

How to use this product

Written for the team. If you did not build the simulator, start here — this is everything you need to open it, drive it, and explain it without guessing.

DOCUMENT	Operator's guide
APPLIES TO	Firmware v1.0.0
HARDWARE	BPV-1 rev.A
MASTER PASSWORD	Vault@2026
TEAM	4 – see §10

CONTENTS	
01 What this is	02 Opening it
03 First run	04 The controls
05 What's on screen	06 Doing things
07 The attack lab	08 The security story
09 Troubleshooting	10 Splitting the demo
11 Glossary	12 Where things live

01 WHAT YOU ARE LOOKING AT

Our hardware project is a pocket-sized ESP32 device with a small OLED screen and a rotary knob. It stores your passwords encrypted in its flash memory. You unlock it with one master password, scroll to the account you want, click, and the device types your username and password into your computer over Bluetooth — the computer thinks an ordinary wireless keyboard is doing the typing.

The board is not built yet. This is the firmware, running in a browser.

That is not a cop-out, and it matters that everyone on the team says it the same way:

“The state machine, the storage format, the display driver, the Bluetooth HID transport and **all of the cryptography** are the real implementations. Only the chip and the radio are simulated.”

Concretely, when you use this thing:

- Keys really are derived with PBKDF2 and HKDF, by the browser's own crypto engine.
- Credentials really are encrypted with AES-256-GCM, and really are decrypted before use.
- The simulated flash really persists — close the tab, come back, your vault is still there.

- Corrupt one byte of a stored credential and decryption really does fail.

Nothing is a picture of a thing. Everything is the thing.

02 OPENING IT

Three ways, all equivalent:

The hosted site	Open the Netlify link. Best for showing someone on their own laptop or phone.
The file	Double-click <code>index.html</code> . No internet needed at all.
A local server	Run <code>serve.bat</code> (Windows) or <code>serve.sh</code> (Mac/Linux). Only needed if a browser refuses to run the crypto from a file — the page tells you on screen if that happens.

Use Chrome or Firefox. There is no install, no build step, no npm, no login.

CAUTION

Keep the browser tab in front while you use it. Browsers slow down background tabs to one tick per second, which makes the typing animation crawl. That is a browser rule, not a bug in our code.

03 FIRST RUN

On a machine that has never opened it, the device boots **unprovisioned** — it has no vault yet, exactly like a factory-fresh board. It asks you to set a master password on the OLED. You have two options.

Set one yourself. Type a password (minimum 4 characters), press `Enter`, type it again to confirm. The device generates a random salt, derives the keys, and seeds five demo credentials.

Or click “Quick factory provision” in the *Demo cheat sheet* card on the left. Same result, instantly, using the standard demo password:

MASTER PASSWORD **Vault@2026**

It is always displayed in the cheat sheet card, so nobody has to remember it. The five seeded accounts are GitHub, College Portal, Gmail, Campus WiFi and Bank (demo).

BEFORE THE REVIEW

Click **Factory wipe** to return to a factory-fresh state. Do this if you have been rehearsing, so the reviewer sees the first-boot provisioning flow.

04 THE CONTROLS

The real device has exactly two inputs: a knob you can turn, and a button you make by pushing the knob in. Everything below is a way of doing those two things.

WHAT YOU WANT	ON SCREEN	ON THE KEYBOARD
Turn the knob	Mouse wheel over the knob, or the CW / CCW buttons	↑ ↓
Click the knob	PRESS , or click the knob	Enter or Space
Hold the knob	HOLD , or hold the knob down	Esc
Type the password	—	just type it; Enter submits
Delete a character	Rotate to DEL and press	Backspace

There are also three front-panel buttons — **POWER**, **RESET** and **LOCK NOW** — which do what they say.

ABOUT THE CHARACTER RING

On the real hardware there is no keyboard, so the password is entered by turning the knob through a ring of characters and clicking to accept each one. DEL and OK are entries in that ring. It is authentic, and it is slow — which is why you can also just type. Both work at the same time.

05 WHAT IS ON THE SCREEN

LEFT COLUMN – THE DEVICE

The OLED is drawn pixel by pixel from a real 1-bit framebuffer, the same way the physical panel would be driven. Unlocked, it looks like this:



The inverted row is the current selection. The status bar shows the record count and the auto-lock countdown.

The three LEDs are power, Bluetooth and flash activity. Under the screen you see the current firmware state and the frame counter.

Guided demo → **Play full walkthrough** runs the entire flow by itself with captions along the bottom. This is your safety net: if anything goes wrong while presenting, press it and talk over it.

Demo cheat sheet shows the master password, the KDF settings, how many records exist, and the live failed-attempt count.

MIDDLE COLUMN – THE OUTSIDE WORLD

Paired host — DESKTOP-PC is a fake computer with a fake login page. When the vault types, the characters land here. Tab really moves the cursor from the username box to the password box, and Enter really submits the form.

BLE sniffer shows what someone with a radio scanner parked next to you would capture. With link encryption on (the default) it is meaningless AES-CCM ciphertext. Untick the box and it becomes readable keystrokes — a deliberate demonstration, not a flaw.

USB provisioning console is the tool you would plug the device into to load credentials onto it (§6). **Attack lab** is nine attacks you can run against the live vault (§7).

RIGHT COLUMN – INSTRUMENTATION

Seven tabs. You do not need all of them at once; each answers one specific question.

TAB	ANSWERS THE QUESTION
Serial monitor	What is the firmware doing right now? Same format as a real ESP32 debug log.
Flash (NVS)	What is actually stored on the chip? Colour-coded raw bytes.
Crypto trace	Which cryptographic operation ran, with what inputs, and how long did it take?
SRAM	What secrets are in memory at this instant, and what has been wiped?
State machine	Where are we in the firmware's 16 states?
Self-test	Is the cryptography actually correct? 19 assertions, about 50 ms.
About / docs	The design write-up, inside the app.

Below the tabs, **Device settings (live)** changes the auto-lock timeout, typing speed, PBKDF2 iterations, max attempts and a few switches. Changing the iteration count only takes effect the next time the vault is provisioned or re-keyed — the page tells you so.

06 DOING THINGS – STEP BY STEP

UNLOCK THE VAULT

Type `Vault@2026`, press `Enter`. Takes about a quarter of a second while PBKDF2 runs.

SEND A CREDENTIAL TO THE COMPUTER

1. Turn the knob to the account you want.
2. Press. You are now on that record's detail screen.
3. `SEND TO PC` is already selected — press again.
4. The device asks you to confirm. Press once more for **YES**.
5. Watch the login form fill in.

The confirmation step is deliberate: the device will not type anything without a physical button press, so a compromised computer cannot silently ask it for your passwords.

LOOK AT A CREDENTIAL WITHOUT SENDING IT

On the detail screen, turn to **SHOW ON SCREEN** and press. The username and password appear on the OLED. Press again to hide — which also wipes the decrypted copy out of memory.

ADD A NEW CREDENTIAL

In the USB provisioning console, fill in a label, a username and a password (the **Gen** button makes a random 20-character one), then click **Encrypt & store record**.

The vault must be **unlocked** — if it is locked the console refuses, because the device has no key to encrypt with. Note what happens here: the host tool hands over plaintext, and the *device* encrypts it. The host never sees the vault key.

DELETE A CREDENTIAL

Click **del** on its row in the record table.

CHANGE THE MASTER PASSWORD

Change master password (re-key) in the provisioning console; the vault must be unlocked. It asks for the new password, generates a fresh salt, derives a completely new key set, and re-encrypts every record under it. Watch the crypto trace while it runs — it is a good demonstration of why a key hierarchy matters.

CHANGE DEVICE SETTINGS

Two places, both live: on the device via menu → *** SETTINGS** (turn to an item, press to cycle its value), or in the browser via the **Device settings (live)** card.

LOCK IT

LOCK NOW, or the *** LOCK NOW** menu entry, or hold the knob on the menu screen, or just walk away — it auto-locks after 60 seconds of inactivity by default.

GET OUT OF A LOCKOUT

Five wrong passwords triggers a cooldown, and the cooldown is real — it counts down on the screen. To skip it during a rehearsal, use **Factory wipe** then **Quick factory provision**.

RUN THE SELF-TESTS

Self-test tab → **Run self-test suite**. You want to see **19/19 passed**.

07 THE ATTACK LAB

Nine attacks. Each runs for real against the live vault and prints a verdict underneath.

ATTACK	WHAT HAPPENS
Dump the flash	You see the raw chip contents. No key, no plaintext — ciphertext only.
Brute-force the password	Five guesses, then an escalating lockout.
Power-cycle to reset the counter	Fails. The counter is written before verification.
Flip one bit in a ciphertext	Decryption refused — the authentication tag no longer matches.
Swap two records	Refused. Each ciphertext is cryptographically bound to its own record.
Rename a label	Refused, for the same reason.
Sniff the Bluetooth link	Leaks if you turn link encryption off. With it on, the capture is meaningless.
Scrape the key out of RAM	Partial leak while unlocked. Nothing at all once locked.
Restore the vault	Undoes the tampering so you can carry on.

Two of them leak on purpose. If a reviewer asks about those, the answer is not defensive:

“The key has to exist in RAM or we could not decrypt anything — so we minimise the window instead: short auto-lock, wipe on every exit path, nothing written to flash. And the sniffing one only leaks with link encryption switched off, which is the point of showing it. A threat model where everything is blocked isn't a threat model.”

CAUTION

Always click **Restore the vault** after the tampering attacks, or later steps will fail.

08 UNDERSTANDING THE SECURITY STORY

Enough to answer questions confidently, without having written the code.

The vault key is never stored anywhere. It is recreated from your master password every time you unlock. Flash contains a random salt, a verifier, and the encrypted records — nothing that can decrypt anything.

The verifier is not the key. Both come from your master password, but through different one-way branches. Someone who reads the verifier off the chip cannot work backwards to the encryption key.

Guessing is expensive on purpose. Every attempt runs 150,000 rounds of hashing. One guess is imperceptible; a billion guesses is not.

The failure counter is written *before* the password is checked. This is the piece to be proud of. The obvious implementation checks first and records the failure afterwards — which loses instantly to “guess, then yank the power before it can write.” Ours cannot be rewound. Prove it by reloading the page mid-attack.

Tampering with storage is detected, not just decryption. Each record's encryption is bound to its own ID and label, so an attacker cannot move your bank ciphertext onto a label you would click.

Secrets are wiped, not just dropped. Decrypted passwords are overwritten twice — once with random bytes, once with zeros — as soon as they are finished with. The SRAM tab shows this happening.

What we do not claim. Once a password has been typed into the computer it is ordinary keyboard input, and a keylogger on that computer would see it. No hardware vault can fix that, and we say so rather than pretending otherwise.

09 TROUBLESHOOTING

SYMPTOM	CAUSE AND FIX
Typing animation is painfully slow	The tab is in the background. Bring it to the front.
It locked itself while I was talking	Auto-lock, working as designed. Set it to 5 min in Device settings.
“Vault is locked” when adding a record	Correct behaviour — unlock first.
A record will not decrypt	You ran a tampering attack. Click Restore the vault .
Stuck in a lockout countdown	Wait, or Factory wipe → Quick factory provision.
Big red “Web Crypto API unavailable”	Rare. Run <code>serve.bat</code> / <code>serve.sh</code> and use the localhost link.
Screen is blank and nothing responds	Click POWER — you probably switched the device off.
My records vanished on another machine	The vault lives in that browser's storage. Each browser, and each device, has its own.

10 SPLITTING THE DEMO ACROSS FOUR PEOPLE

A suggestion, not a rule. Roughly six minutes.

WHO	COVERS
1 — Introduction	What the product is, why a hardware vault beats a password manager on the host, the boot sequence and the team splash.
2 — Authentication	Wrong password, the persisted counter, the reload-proof, lockout escalation. The strongest section.
3 — Cryptography	The unlock trace, the key hierarchy, the flash dump, the plaintext scan, the self-test suite.
4 — Operation and threats	Selecting a record, the confirmation, the Bluetooth transmission, zeroization, then two or three attacks.

Whoever is driving the mouse should have `DEMO_SCRIPT.md` open — it has the exact lines and the nine questions we expect.

11 GLOSSARY

Terms that will come up, and short answers you can give without hedging.

PBKDF2	Turns a human password into a proper key by hashing it 150,000 times. The repetition is the point: it makes each guess expensive.
HKDF	Takes one key and produces several independent keys from it. We use it to make a verifier and an encryption key that cannot be derived from one another.
AES-256-GCM	The encryption. "GCM" adds an authentication tag, so it detects tampering instead of silently returning garbage.
Tag	The 16 bytes of proof attached to each encrypted record. Wrong tag, no decryption.
AAD	Associated data — extra information mixed into the tag but not encrypted. We put the record's ID and label there, which is what makes swapping and relabelling detectable.
IV / nonce	12 random bytes used once per encryption, so encrypting the same password twice produces different ciphertext.
Salt	16 random bytes stored alongside the vault, so two devices with the same master password still end up with different keys.
Zeroization	Deliberately overwriting a secret in memory instead of just forgetting about it.
NVS	The ESP32's non-volatile storage, its flash key-value store. Simulated here by the browser's local storage.
HID	The USB/Bluetooth standard for keyboards and mice. Pretending to be a keyboard is how the vault types without any software installed on the computer.
Verifier	The value stored on the chip that lets the device recognise the correct master password without storing the password or the key.

12 WHERE EVERYTHING LIVES

<code>index.html</code>	The whole simulator, one file.
<code>README.md</code>	The design, the threat model, the hardware mapping.
<code>HOW_TO_USE.md</code>	This document.
<code>DEMO_SCRIPT.md</code>	The presentation script and expected questions.
<code>netlify.toml</code>	Hosting configuration.
<code>preview.png</code>	Link-preview image.
<code>serve.bat</code> / <code>serve.sh</code>	Backup launchers.

Repository: github.com/vatsal-agra/Bluetooth-Password-Vault